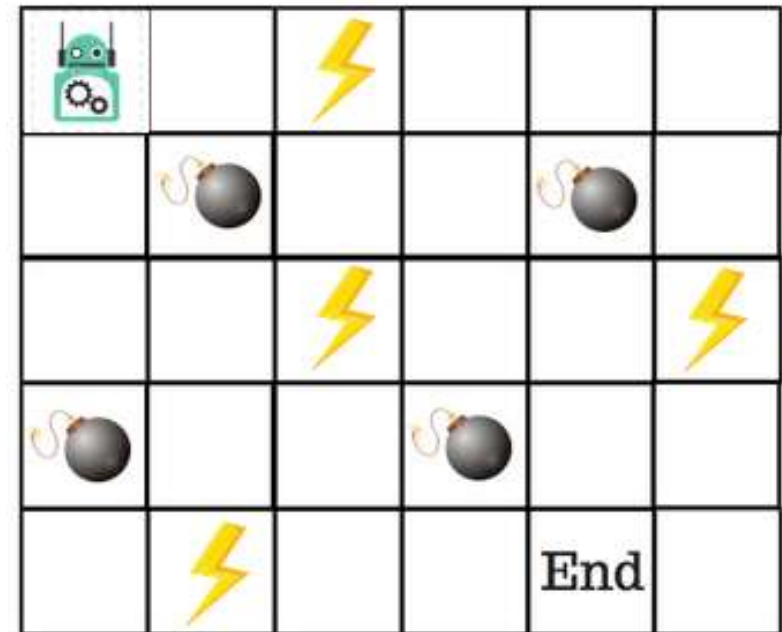


Las bases del aprendizaje por refuerzo

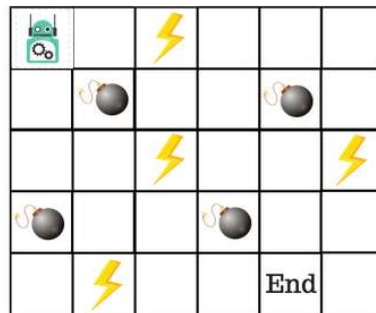
Versión simplificada Q-Learning

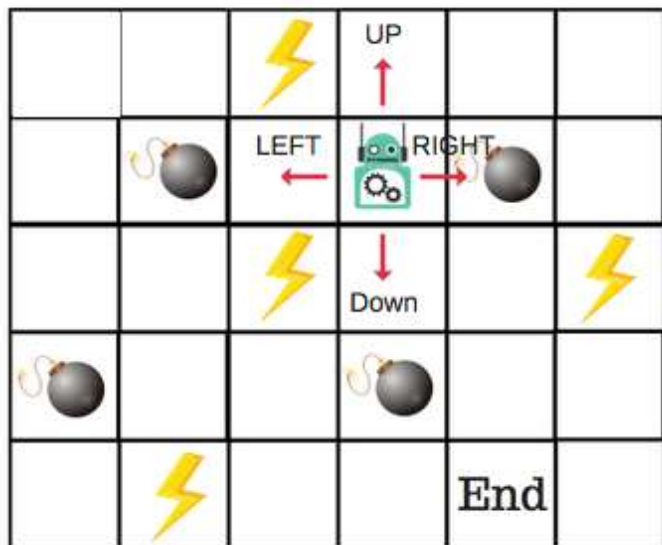
- Digamos que un **robot** tiene que cruzar un **laberinto** y llegar al punto final. Hay **minas**, y el robot sólo puede moverse una casilla a la vez. Si el robot pisa una mina, el robot muere. El robot tiene que llegar al punto final en el menor tiempo posible



El sistema de puntuación/recompensa es el siguiente:

- 1.El robot pierde 1 punto en cada paso. Esto se hace para que el robot tome el camino más corto y llegue a la meta lo más rápido posible.
- 2.Si el robot pisa una mina, la pérdida de puntos es de 100 y el juego termina.
- 3.Si el robot obtiene poder ⚡ , gana 1 punto.
- 4.Si el robot alcanza la meta, el robot obtiene 100 puntos.





¿Cómo entrenamos a un robot para llegar a la meta final con el camino más corto sin pisar una mina?

- **Introducción a Q-Table**

- Q-Table es sólo un nombre elegante para una simple tabla de búsqueda donde calculamos las máximas recompensas futuras esperadas por acción en cada estado. Básicamente, esta tabla nos guiará a la mejor acción en cada estado.

Acciones para construir Q-Table

- Habrá cuatro números de acciones en cada mosaico sin bordes. Cuando un robot está en un estado puede moverse hacia arriba, hacia abajo, hacia la derecha o la izquierda.

Actions : ↑ → ↓ ←

Start				
Nothing / Blank				
Power				
Mines				
END				

En la Q-Table:

- Las columnas son las acciones
- Las filas son los estados.

Acciones para construir Q-Table

- Habrá cuatro números de acciones en cada mosaico sin bordes. Cuando un robot está en un estado puede moverse hacia arriba, hacia abajo, hacia la derecha o la izquierda.

Actions : ↑ → ↓ ←

Start				
Nothing / Blank				
Power				
Mines				
END				

En la Q-Table:

- Las columnas son las acciones
- Las filas son los estados.

Acciones para construir Q-Table (II)

Cada puntuación de la Q-Table será la máxima recompensa futura esperada que el robot recibirá si toma esa acción en ese estado. Se trata de un proceso iterativo, ya que necesitamos mejorar la Q-Table en cada iteración.

Pero las preguntas son:

- **¿Cómo calculamos los valores de la Q-Table?**
- **¿Los valores están disponibles o predefinido**

Actions : ↑ → ↓ ←

Start				
Nothing / Blank				
Power				
Mines				
END				

En la Q-Table:

- Las columnas son las acciones
- Las filas son los estados.

ALGORITMO Q-LEARNING

Q-function

- La Q-function utiliza la ecuación de Bellman y toma dos entradas: estado (s) y acción (a).

$$Q^{\pi}(s_t, a_t) = \underline{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | s_t, a_t]$$

Q-Values for the state given a particular state

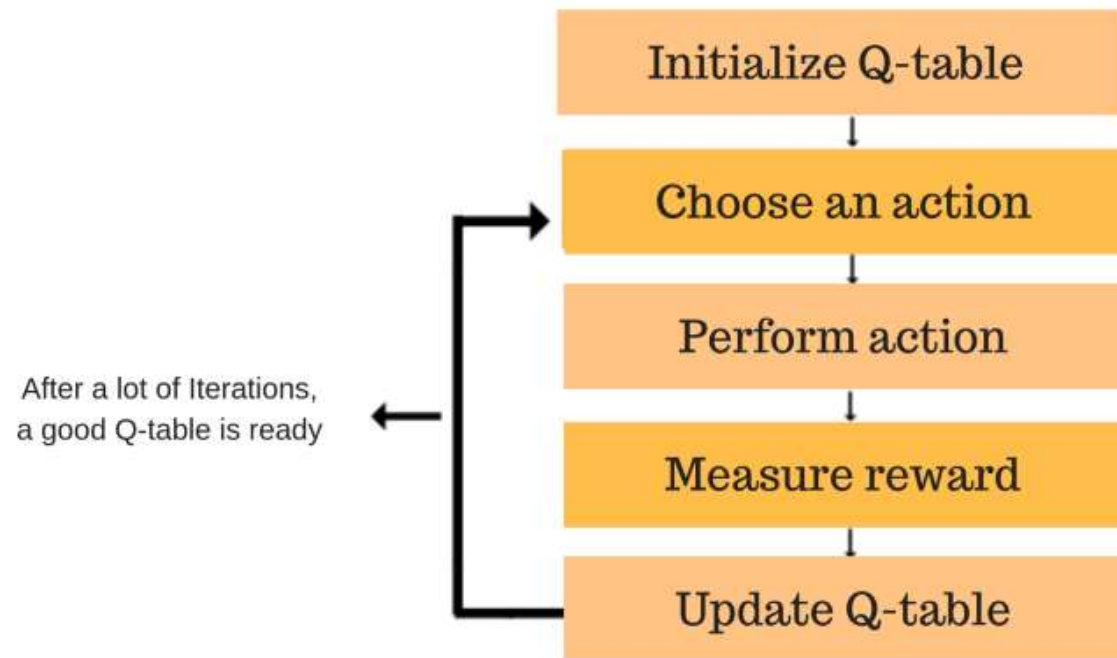
Expected discounted cumulative reward

Given the state and action

Usando la función anterior, obtenemos los valores de **Q** para las celdas de la tabla.
Cuando empezamos, todos los valores de la Q-Table son ceros.

ALGORITMO Q-LEARNING

- **Introducción al proceso del algoritmo Q-Learning**



la Q-function nos da mejores y mejores aproximaciones, actualizando continuamente los Q-values de la tabla.

ALGORITMO Q-LEARNING

- **Paso 1. Inicializar la Tabla Q-Table**

Actions :    

Start	0	0	0	0
Nothing / Blank	0	0	0	0
Power	0	0	0	0
Mines	0	0	0	0
END	0	0	0	0

Primero construiremos una Q-Table:

- Hay n columnas, donde n= número de acciones.
- Hay m filas, donde m= número de estados.
- Inicializaremos los valores en 0.

					
					
					
					
				End	

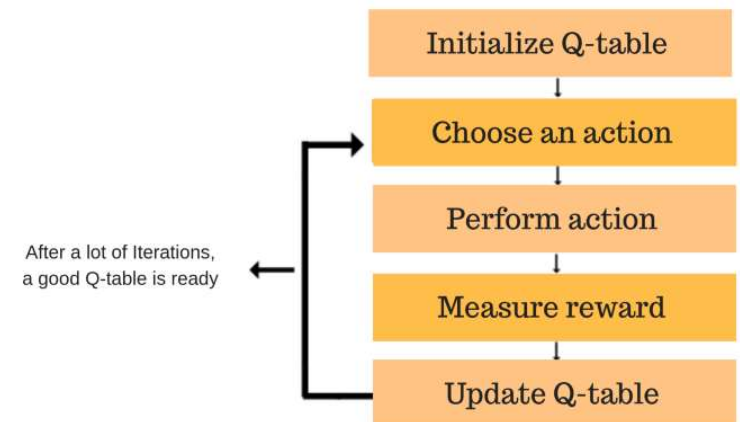
ALGORITMO Q-LEARNING

• Paso 2 y 3. Elegir acción a realizar

- Al comienzo todo está en Cero. Vamos a utilizar algo llamado la **estrategia codiciosa de Épsilon**.
- Al principio, las tasas de épsilon serán más altas. El robot explorará el entorno y elegirá acciones al azar. La lógica detrás de esto es que el robot no sabe nada sobre el medio ambiente.
- A medida que el robot explora el entorno, la velocidad de épsilon disminuye y el robot comienza a explotar el entorno.
- Durante el proceso de exploración, el robot adquiere progresivamente más confianza en la estimación de los Q-values.

Esta combinación de pasos se realiza por un tiempo indefinido.

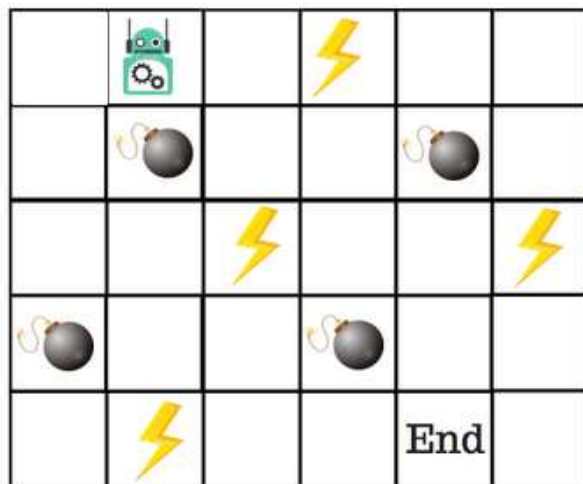
Esto significa que este paso se ejecuta hasta el momento en que detenemos el entrenamiento, o el bucle de entrenamiento se detiene como se define en el código.



ALGORITMO Q-LEARNING

- **Paso 2, 3 (Continuación)**

Inicio, Robot no sabe nada, por ejemplo acción aleatoria -- Derecha



Actions :

Start	0	0	0	0
Nothing / Blank	0	0	0	0
Power	0	0	0	0
Mines	0	0	0	0
END	0	0	0	0

ALGORITMO Q-LEARNING

- Paso 4, 5

Ahora hemos tomado una acción y observado un resultado y recompensa. Necesitamos actualizar la función $Q(s,a)$.

$$\text{New } Q(s,a) = Q(s,a) + \alpha [R(s,a) + \gamma \max_{a'} Q'(s',a') - Q(s,a)]$$

- New Q Value for that state and the action
- Learning Rate
- Reward for taking that action at that state
- Current Q Values
- Maximum expected future reward given the new state (s') and all possible actions at that new state.
- Discount Rate

En el caso del juego del robot, para reiterar la estructura de puntuación/recompensa es:

- **power** = +1
- **mine** = -100
- **end** = +100

ALGORITMO Q-LEARNING

- Paso 4, 5

Ahora hemos tomado una acción y observado un resultado y recompensa. Necesitamos actualizar la función $Q(s,a)$.

$$\text{New } Q(s,a) = Q(s,a) + \alpha [R(s,a) + \gamma \max_{a'} Q'(s',a') - Q(s,a)]$$

- New Q Value for that state and the action
- Learning Rate
- Reward for taking that action at that state
- Current Q Values
- Maximum expected future reward given the new state (s') and all possible actions at that new state.
- Discount Rate

En el caso del juego del robot, para reiterar la estructura de puntuación/recompensa es:

- **power** = +1
- **mine** = -100
- **end** = +100